

Izveštaj - obrazloženje rešenja - Faza 2

Projekat "Atoll" - veštačka inteligencija

Tim "X-Files"

Tara Milijić

Nikola Ristić

Nikola Ristovski

2025/26

Provera kraja igre

Nikola Ristovski

Implementacija provere kraja igre je ključni problem kada govorimo o bilo kojoj društvenoj igri. S obzirom da nam je cilj da povežemo 2 ostrva, uz ograničenja o broju ostrvaca koji su "obuhvaćeni" tim povezivanjem, potrebno nam je da imamo informacije o samim ostrvima. To u našem projektu realizujemo pomoćnom funkcijom `get_all_islands(board_size: int)` koja nam kreira pomoćnu strukturu `islands`. Ona ima sledeću strukturu:

```
{
  ('GREEN', 0) : {
    'NEIGHBORS' : ["Koja ostrva su susedi ovom ostrvu", npr ('RED', 0)],
    'CELLS': ["Koje celije pripadaju datom ostrvu", npr ('C', 0), ('D', 0)]
  },
  ...
}
```

Ova struktura pamti, kao što se vidi iz primera, ulančavanje ostrvaca, dakle GREEN 0 ostrvo je povezano sa RED 0 i RED 1, pa je RED 1 povezano sa GREEN 0 i GREEN 1 itd. Ovo je ključno kasnije kada treba da izbrojimo koliko ostrvaca je obuhvaćeno povezivanjem 2 ostrvaca iste boje. Tada ćemo jednostavno ići od suseda do suseda, počevši od prvog, a sve do drugog ostrva koje smo uspešno povezali, i brojati susede kroz koje prolazimo. Ideja je da ukoliko je taj broj veći ili jednak 7, došli smo do stanja gde je prepoznata pobeda onog igrača koji je spojio ta 2, početno i krajnje ostrvo iste boje. Broj 7 je statički, odnosno uvek je threshold za proveru pobede jednak 7 obuhvaćenih ostrvaca, jer bez obzira na `board_size`, uvek imamo tačno 12 ostrvaca - 6 GREEN i 6 RED.

Funkcija za kreiranje ove pomoćne strukture kreira prvo ostrva GREENm i REDm, m ide od 0 do 6, zajedno sa njihovim susedima, dakle GREEN ima susede RED sa obe strane, a RED ima GREEN susede sa obe strane. Nakon toga potrebno je da dodelimo svakom ostrvu one ćelije sa naše table koje ga sačinjavaju. To radimo slično kao u funkciji za inicijalizaciju ostrva iz prethodne faze, i određene ćelije pridružujemo određenom ostrvu. Primećuje se da je ovo delom hard-coded. To je zato što po pravilima igre koju kreiramo za ovaj projekat uvek važi da imamo tačno 12 ostrva, a i uvek znamo gde se ona nalaze jedna u odnosu na druge.

Još jedna funkcija koja nam je pomoćna je `map_cells_to_islands(islands)` koja radi obrnut postupak: Za svaku ćeliju koja sačinjava neko ostrvo, ona mapira njega na to ostrvo koje on sačinjava. Ovo je ključno kasnije kada proveravamo da li je trenutna ćelija u postupku traženja deo nekog ostrva ili ne.

Centralna funkcija koja određuje da li su neka dva obodna ostrva iste boje (ona obeležena sa GREEN 0, RED 3 itd.) povezana je `find_connected_islands(state, islands)`. To vršimo pomoću BFS pretrage. Formiramo `island_connections` rečnik koji za svako obodno ostrvo kreira skup ostrvaca sa kojima je on povezan (inicijalno - svaki prazan). U postupku traženja klasičnim BFS-om, krećemo pretragu od svake ćelije svakog ostrva. Proveravamo da li je neki od suseda te ćelije (polja) iste boje kao i trenutna ćelija. Ako jeste, ona nastavlja taj put. Istovremeno, proveravamo da li je trenutna ćelija jedna od onih koje sačinjavaju neki od obodnih ostrvaca, a pritom iste boje kao i put koji trenutno posmatramo. Ako jeste, to znači da smo uspešno povezali 2 ostrva iste boje. To označimo dodavanjem dostignutog ostrva u skup trenutnog ostrva od koga smo započeli pretragu. Rezultat ovog procesa jeste upravo rečnik gde je zapisano sa kojim je sve ostrvima povezano svako od naših ostrva.

Ako se desi da smo povezali nekoliko ostrvaca u putu, recimo GREEN 0 smo povezali sa GREEN 1, pa GREEN 1 povezali sa GREEN 2, to znači da imamo zapravo komponentu od 3 povezana ostrva. To praktično znači da je GREEN 0 povezan sa GREEN 2 preko GREEN 1, i to produžava put koji smo formirali. Formiranje takvih komponenti ostvarujemo pomoćnom funkcijom `get_islands_components(connections, start)`. Ovu ćemo funkciju pozvati za svaki od ostrvaca kasnije u glavnoj funkciji za proveru kraja igre.

Poslednja pomoćna funkcija je `island_distance(islands, start, target)`. Ova funkcija obezbeđuje određivanje broja ostrva koja su obuhvaćena povezivanjem 2 istobojna ostrva. S obzirom da imamo tablu takvu da ostrva idu u krug, možemo brojati ostrva u smeru kazaljke na satu i obrnuto smeru kazaljke na satu. Ideja je da u obzir uzmemo samo minimum od ta 2 broja, po pravilima igre. Zato, kao što je i ranije

objašnjeno, krećemo od prvog, start, ostrva, i brojimo skokove od tog ostrva do drugog, target, ostrva koje smo medjusobno kamenčićima povezali. Skaćemo od suseda do suseda koristeći pomoćnu islands strukturu. Prvo brojimo u clockwise, a zatim i u counter-clockwise smeru i kao rezultat funkcije vraćamo min između ta 2.

Glavna funkcija je `check_win_condition(state, board_size)`. Ona se služi prethodno opisanim funkcijama. Prvo kreiramo pomoćnu strukturu sa `get_all_islands(board_size)`. Zatim odredimo i koja ostrva su povezana uz pomoć `find_connected_islands(state, islands)`. Nakon toga, za svako ostrvo koje imamo u strukturi, pronađemo komponente sa kojima je povezan koristeći `get_island_components(connections, island)`. Preskaćemo slučaj kada je dužina komponente manja od 2, jer to znači da ostrvo uopšte nije povezano ni sa jednim drugim. Nakon toga za svaki od ostrva u komponenti za trenutno posmatrano ostrvo odredimo dužinu između njih pomoću `island_distance(islands, island, other)`, što zapravo predstavlja broj ostrvaca koje smo obuhvatili spojem ta 2 ostrva. Ako je taj broj veći ili jednak broju threshold koji je uvek 7 za našu igru, otkrili smo pobedu. Pobednik je određen bojom ostrva koja smo povezali.

Ovu funkciju pozivamo nakon svakog unosa poteza od strane bilo kog od igrača.

Funkcije za odigravanje partije

Tara Milijić

Implementacija funkcija za odigravanje partije obuhvata upravljanje stanjem igre, obradu poteza igrača i prikaz igre kroz grafički interfejs. Sistem je organizovan modularno kroz nekoliko ključnih komponenti.

Globalno stanje igre održavamo kroz promenljivu `current_player` koja čuva informaciju o tome koji igrač je trenutno na potezu ("GREEN" ili "RED"). `set_current_player(player)` postavlja trenutnog igrača, `get_current_player()` vraća trenutnog igrača i `toggle_current_player()` menja trenutnog igrača i vraća novog igrača na potezu. Ove funkcije obezbeđuju kontrolisano menjanje stanja i sprečavaju direktan pristup globalnoj promenljivoj iz drugih delova koda.

Glavna funkcija za izvršavanje poteza je `set_a_cell(state, position, current_player)`. Ova funkcija prihvata trenutno stanje table, poziciju na kojoj se potez igra i boju igrača koji igra. Funkcija ažurira stanje table tako što postavlja boju igrača na zadatu poziciju (`state[position][0] = current_player`) i vraća izmenjeno stanje.

Funkcija `handle_click(cell, canvas, state)` predstavlja glavnu logiku za obradu korisničkog klika na tablu. Globalne promenljive `clicked_cell`, `click_job` i `can_play` koriste se za praćenje kliknutog polja, odloženog posla za resetovanje klika i dozvole za igranje.

Prvo se proverava da li je dozvoljeno igranje (`can_play`). Ako nije, funkcija se odmah vraća bez izvršavanja poteza. Validnost poteza proveravamo funkcijom `is_move_valid(state, cell)`. Ukoliko je potez validan, izvršava se potez pozivom `set_a_cell(state, cell, current_player)`. Ispisuje se informacija o odigranom potezu i menja se igrač na potezu pozivom `toggle_current_player()`. Na kraju svakog poteza poziva se `check_win_condition(state, game_config["board_size"])` kako bi se proverilo da li je igra završena. Ako postoji pobednik, postavlja se `can_play = False` kako bi se sprečili dalje potezi i prikazuje se poruka o pobedi kroz informacioni dijalog.

Funkcija `draw(state)` inicijalizuje grafički interfejs i pokreće igru. Ona prvo postavlja početnog igrača na osnovu konfiguracije (`green_plays_first`), kreira glavni prozor sa zadatim naslovom i rezolucijom, učitava resurse i kreira canvas za crtanje.

Operator promene stanja

Nikola Ristić

U ovoj fazi projekta, fokus je bio na kreiranju mehanizama koji omogućavaju promenu stanja table, kao i predviđanje svih mogućih budućih stanja, što je osnov za implementaciju algoritama veštačke inteligencije u narednim fazama.

Osnovna je funkcija `get_next_state(current_state, move, player)`. Njena uloga je da simulira odigravanje poteza bez modifikacije originalnog (trenutnog) stanja table. Koristi se `copy.deepcopy(current_state)` kako bi se kreirala potpuno nova instanca table. Ovo je od ključnog značaja za algoritme pretrage (poput min-max algoritma), jer omogućava “gledanje unapred” bez narušavanja trenutne strukture table. Funkcija mapira koordinatu poteza u rečnik stanja i postavlja indikator igrača na to polje.

Da bi AI mogao da donese odluku, on mora da poznaje sve dostupne opcije. To se postiže kroz dve povezane funkcije. Prva je `get_all_possible_moves(current_state)` koja prolazi kroz rečnih stanja i filtrira sve koordinate gde je trenutno stanje polja `None`. Rezultat je lista svih slobodnih ćelija na tabli. Funkcija `get_all_possible_states(current_state, player)` predstavlja generator skupa svih mogućih ishoda za sledeći potez. Ona prolazi kroz sve slobodne koordinate i za svaku poziva `get_next_state`. Na taj način dobijamo listu novih rečnika stanja koji predstavljaju “decu” trenutnog čvora u stablu pretrage.

Kao most između grafičkog interfejsa i logike modificovana je funkcija `handle_click(cell, canvas, state)` koja služi kao most između grafičkog interfejsa i logike igre. Program nakon načinjenog validnog poteza ispisuje broj preostalih slobodnih polja u konzoli i listu koordinata sledećih dostupnih poteza, što olakšava debugovanje i praćenje dinamike igre.